

Data Structures Home Work Set 2

Register No: 2025115058

Name: Sri Senthil Adavan K

Q1. Define an integer array. Declare an integer pointer and initialize it to point to the first element of the array. Read and print the array elements using pointer arithmetic and modify the elements using pointer arithmetic.

Code:

```
#include <stdio.h>
```

```
int main()
{
    int a[10], n, i;
    int *p;
```

```
    printf("Enter number of elements: ");
    scanf("%d", &n);
```

```
    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }
```

```
    p = a;
```

```
    printf("Array elements are:\n");
    for(i = 0; i < n; i++)
    {
        printf("%d ", *(p + i));
    }
```

```
    printf("\nEnter value to add to all elements: ");
    scanf("%d", &n);
```

```
    for(i = 0; i < 10; i++)
    {
        if(i < 5)
```

```

{
*(p + i) = *(p + i) + n;
}
}

printf("Modified array:\n");
for(i = 0; i < 10; i++)
{
printf("%d ", *(p + i));
}

return 0;
}

```

Output:

```

Enter number of elements: 3
Enter elements:
1
4
7
Array elements are:
1 4 7
Enter value to add to all elements: 2
Modified array:
3 6 9 2 11 9 5 3 10 10 23 6 14 -2026 704032 29805 -171065 1312 32766

```

Q2. Calculate the string length using a pointer.

Code:

```

#include <stdio.h>

int main()
{
char str[100];
char *p;
int length = 0;

printf("Enter a string: ");
fgets(str, sizeof(str), stdin);

p = str;

while(*p != '\0')
{
if(*p != '\n')
{

```

```

length++;
}
p++;
}

printf("Length of string = %d", length);

return 0;
}

```

Output:

```

Enter a string: ssak
Length of string = 4

```

Q3. Write a function that calculates both the sum and product of two numbers. Return both results to the calling function using pointer arguments.

Code:

```

#include <stdio.h>

void calculate(int a, int b, int *sum, int *product)
{
    *sum = a + b;
    *product = a * b;
}

int main()
{
    int a, b;
    int sum, product;

    printf("Enter two numbers: ");
    scanf("%d%d", &a, &b);

    calculate(a, b, &sum, &product);

    printf("Sum = %d\n", sum);
    printf("Product = %d\n", product);

    return 0;
}

```

Output:

Enter two numbers: 20
21
Sum = 41
Product = 420

Q4. Create an array of structures. Use dynamic memory allocation to allocate memory space and use a structure pointer to iterate and access/modify the members.

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Student
{
    int id;
    char name[50];
    float mark;
};

int main()
{
    struct Student *p;
    int n, i;

    printf("Enter number of students: ");
    scanf("%d", &n);

    p = (struct Student *)malloc(n * sizeof(struct Student));

    if(p == NULL)
    {
        printf("Memory allocation failed");
        return 0;
    }

    for(i = 0; i < n; i++)
    {
        printf("Enter student %d id: ", i + 1);
        scanf("%d", &(p + i)->id);

        printf("Enter name: ");
        scanf("%s", (p + i)->name);

        printf("Enter mark: ");
        scanf("%f", &(p + i)->mark);
    }
```

```

printf("\nStudent details:\n");

for(i = 0; i < n; i++)
{
printf("ID = %d\n", (p + i)->id);
printf("Name = %s\n", (p + i)->name);
printf("Mark = %.2f\n", (p + i)->mark);
}

printf("\nEnter new mark for each student:\n");

for(i = 0; i < n; i++)
{
printf("Enter new mark for %s: ", (p + i)->name);
scanf("%f", &(p + i)->mark);
}

printf("\nModified details:\n");

for(i = 0; i < n; i++)
{
printf("%d %s %.2f\n", (p + i)->id, (p + i)->name, (p + i)->mark);
}

free(p);

return 0;
}

```

Output:

```

Enter number of students: 3
Enter student 1 id: 202511
Enter name: SSAK
Enter mark: 100
Enter student 2 id: 202512
Enter name: sak
Enter mark:99
Enter student 3 id: 202513
Enter name: sk
Enter mark: 98

```

```

Student details:
ID = 202511
Name = SSAK
Mark = 100.00
ID = 202512
Name = sak
Mark = 99.00

```

ID = 202513
Name = sk
Mark = 98.00

Enter new mark for each student:
Enter new mark for SSAK: 100
Enter new mark for sak: 90
Enter new mark for sk: 43

Modified details:
202511 SSAK 100.00
202512 sak 90.00
202513 sk 43.00

Q5. Define two functions with the same signature. Declare a function pointer and use it to call both functions.

Code:

```
#include <stdio.h>

void add(int a, int b)
{
    printf("Sum = %d\n", a + b);
}

void multiply(int a, int b)
{
    printf("Product = %d\n", a * b);
}

int main()
{
    int a, b;
    void (*fp)(int, int);

    printf("Enter two numbers: ");
    scanf("%d%d", &a, &b);

    fp = add;
    fp(a, b);

    fp = multiply;
    fp(a, b);

    return 0;
}
```

Output:

Enter two numbers: 48
43
Sum = 91
Product = 2064

Q6. Declare an integer, a pointer to it and a double pointer. Print the values and addresses at each level.

Code:

```
#include <stdio.h>

int main()
{
    int a;
    int *p;
    int **q;

    printf("Enter an integer: ");
    scanf("%d", &a);

    p = &a;
    q = &p;

    printf("Value of a = %d\n", a);
    printf("Address of a = %p\n", (void *)&a);

    printf("Value of p = %p\n", (void *)p);
    printf("Address of p = %p\n", (void *)&p);
    printf("Value pointed by p = %d\n", *p);

    printf("Value of q = %p\n", (void *)q);
    printf("Address of q = %p\n", (void *)&q);
    printf("Value pointed by q = %p\n", (void *)*q);
    printf("Value of a using double pointer = %d\n", **q);

    return 0;
}
```

Output:

Enter an integer: 3
Value of a = 3
Address of a = 0x7ffc949be91c
Value of p = 0x7ffc949be91c

Address of p = 0x7ffc949be910
Value pointed by p = 3
Value of q = 0x7ffc949be910
Address of q = 0x7ffc949be908
Value pointed by q = 0x7ffc949be91c
Value of a using double pointer = 3

Q7. Sort an array of floating-point numbers using Selection Sort.

Code:

```
#include <stdio.h>
```

```
int main()
```

```
{  
float a[50], temp;  
int n, i, j, min;
```

```
printf("Enter number of elements: ");  
scanf("%d", &n);
```

```
printf("Enter elements:\n");  
for(i = 0; i < n; i++)  
{  
scanf("%f", &a[i]);  
}
```

```
for(i = 0; i < n - 1; i++)  
{  
min = i;
```

```
for(j = i + 1; j < n; j++)  
{  
if(a[j] < a[min])  
{  
min = j;  
}  
}
```

```
temp = a[i];  
a[i] = a[min];  
a[min] = temp;  
}
```

```
printf("Sorted array:\n");
```

```
for(i = 0; i < n; i++)  
{  
printf("%.2f ", a[i]);
```



```
}  
  
return 0;  
}
```

Output:

```
Enter number of elements: 4  
Enter elements:  
23  
5  
12  
9  
Sorted array:  
5.00 9.00 12.00 23.00
```

Q8. Define a structure Student with id, name and grade. Create an array of Student structures and sort them based on different fields.

Code:

```
#include <stdio.h>  
#include <string.h>  
  
struct Student  
{  
    int id;  
    char name[50];  
    float grade;  
};  
  
int main()  
{  
    struct Student s[50], temp;  
    int n, i, j, choice;  
  
    printf("Enter number of students: ");  
    scanf("%d", &n);  
  
    for(i = 0; i < n; i++)  
    {  
        printf("\nEnter details of student %d\n", i + 1);  
  
        printf("ID: ");  
        scanf("%d", &s[i].id);  
  
        printf("Name: ");  
        scanf("%s", s[i].name);
```

```
printf("Grade: ");
scanf("%f", &s[i].grade);
}
```

```
printf("\nSort by:\n");
printf("1. ID\n");
printf("2. Name\n");
printf("3. Grade\n");
printf("Enter your choice: ");
scanf("%d", &choice);
```

```
for(i = 0; i < n - 1; i++)
{
    for(j = i + 1; j < n; j++)
    {
        if(choice == 1)
        {
            if(s[i].id > s[j].id)
            {
                temp = s[i];
                s[i] = s[j];
                s[j] = temp;
            }
        }
    }
}
```

```
if(choice == 2)
{
    if(strcmp(s[i].name, s[j].name) > 0)
    {
        temp = s[i];
        s[i] = s[j];
        s[j] = temp;
    }
}
```

```
if(choice == 3)
{
    if(s[i].grade > s[j].grade)
    {
        temp = s[i];
        s[i] = s[j];
        s[j] = temp;
    }
}
}
```

```
printf("\nSorted student details:\n");
```

```

for(i = 0; i < n; i++)
{
printf("ID: %d Name: %s Grade: %.2f\n", s[i].id, s[i].name, s[i].grade);
}

return 0;
}

```

Output:

Enter number of students: 2

Enter details of student 1

ID: 121

Name: SSAK

Grade: 99

Enter details of student 2

ID: 1234

Name: adavan

Grade: 86

Sort by:

1. ID

2. Name

3. Grade

Enter your choice: 2

Sorted student details:

ID: 1234 Name: Adavan Grade: 86.00

ID: 121 Name: SSAK Grade: 99.0

Q9. Sort a set of negative numbers using Radix Sort.

Code:

```
#include <stdio.h>
```

```
void sortPositive(int a[], int n, int place)
```

```
{
```

```
int output[50];
```

```
int count[10] = {0};
```

```
int i;
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
count[(a[i] / place) % 10]++;
```

```
}
```

```

for(i = 1; i < 10; i++)
{
count[i] = count[i] + count[i - 1];
}

for(i = n - 1; i >= 0; i--)
{
output[count[(a[i] / place) % 10] - 1] = a[i];
count[(a[i] / place) % 10]--;
}

for(i = 0; i < n; i++)
{
a[i] = output[i];
}
}

void radixSortPositive(int a[], int n)
{
int i, max, place;

max = a[0];

for(i = 1; i < n; i++)
{
if(a[i] > max)
{
max = a[i];
}
}

for(place = 1; max / place > 0; place = place * 10)
{
sortPositive(a, n, place);
}
}

int main()
{
int a[50], positive[50], negative[50];
int n, i;
int pcount = 0, ncount = 0;

printf("Enter number of elements: ");
scanf("%d", &n);

printf("Enter elements:\n");

for(i = 0; i < n; i++)

```

```

{
scanf("%d", &a[i]);

if(a[i] < 0)
{
negative[ncount] = -a[i];
ncount++;
}
else
{
positive[pcount] = a[i];
pcount++;
}
}

if(ncount > 0)
{
radixSortPositive(negative, ncount);
}

if(pcount > 0)
{
radixSortPositive(positive, pcount);
}

printf("Sorted array:\n");

for(i = ncount - 1; i >= 0; i--)
{
printf("%d ", -negative[i]);
}

for(i = 0; i < pcount; i++)
{
printf("%d ", positive[i]);
}

return 0;
}

```

Output:

Enter number of elements: 4

Enter elements:

23

73

65

08

Sorted array:

8 23 65 73

Q10. Write a C program to copy one array to another using pointers.

Code:

```
#include <stdio.h>

int main()
{
    int a[50], b[50];
    int n, i;
    int *p, *q;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements of first array:\n");

    for(i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    p = a;
    q = b;

    for(i = 0; i < n; i++)
    {
        *(q + i) = *(p + i);
    }

    printf("First array:\n");

    for(i = 0; i < n; i++)
    {
        printf("%d ", *(p + i));
    }

    printf("\nCopied array:\n");

    for(i = 0; i < n; i++)
    {
        printf("%d ", *(q + i));
    }

    return 0;
}
```

Output:

Enter number of elements: 4

Enter elements of first array:

43

48

98

56

First array:

43 48 98 56

Copied array:

43 48 98 56